

Lab 3 - Inheritance, Polymorphism, and Class Hierarchies

Emily Antosch · Furkan Yildirim · Julian Moldenhauer
HAW Hamburg · TI

This lab introduces inheritance and advanced object-oriented programming concepts through three progressive tasks. Students will learn class inheritance by extending the Person class to create a Student class, practice polymorphism and method overriding, and work with scalable data structures for realistic applications. The exercises progress from student management systems to vehicle hierarchies and animal classification, emphasizing inheritance relationships, code reusability, and polymorphic behavior.

Tasks

Task 1: Student Management with Inheritance	2
Task 2: Vehicle Hierarchy with Polymorphism	8
Task 3: Animal Classification System	13
Task 4: Collections in Java Programming	20
Task 5: Design Patterns Implementation	28
Task 6: Exception Handling and File I/O	36
Lab Execution:	46

Submission Deadline

Deadline to upload the solutions for all tasks is Saturday, 11:59 pm before the lab date.

General Information

The following tasks are to be worked on in fixed teams of two. Each team member must be able to explain all solutions. Please submit only one solution for each team of two. The submission must be a PDF file in our Moodle room with the name and matriculation number. Solutions must be in digital format with intermediate steps and detailed explanations (no handwritten scans). You can use any tool or drawing program of your choice to create the diagrams. If you have questions or need support, use the forum in our Moodle room and help each other.

Task 1: Student Management with Inheritance

Extend your Java program from Lab 2 to demonstrate inheritance by creating a Student class that inherits from the Person class. This task introduces class inheritance, scalability, and advanced object management.

Extend your name management program as follows:

- Create a new Student class that inherits all aspects of the Person class
- Add a matriculation number attribute that starts at 1001 and increments automatically
- Use a constant MAX_ANZAHL to define maximum students (default: 500, scalable to millions)
- Implement proper encapsulation for the matriculation number (private with getter/setter)
- Extend the menu system:
 - Input 0: Exit program
 - Input MAX_ANZAHL+1: Display all student attributes
 - Input MAX_ANZAHL+2: Display maximum manageable students

Detailed requirements:

- Create a new class Students that inherits all aspects of the Person class in addition to the matriculation number. The matriculation number starts with 1001 and increases by one with each additional person. This matriculation number can also only be accessed via appropriate methods. Otherwise, everything remains as before, i.e., the program is terminated with input 0, with input MAX_ANZAHL+1 all attributes of all students are displayed, and with MAX_ANZAHL+2 the maximum number of students manageable by the program is output.

Preparation

First clarify the task by drawing the Student and Person classes according to UML notation, considering their inheritance relationship. Then define all necessary classes, methods, and variables in Java.

Assistance

The following code snippets demonstrate key inheritance concepts:

Basic inheritance structure:

```
1      public class Person {
2          protected String firstName;
3          protected String lastName;
4          protected int day, month, year;
5          private static int personCount = 0;
6
7          public Person(String firstName, String lastName) {
8              this.firstName = firstName;
9              this.lastName = lastName;
10             personCount++;
11         }
12
13         // Getter and setter methods...
14     }
15
16     public class Student extends Person {
17         private int matriculationNumber;
18         private static int nextMatriculationNumber = 1001;
19
20         public Student(String firstName, String lastName) {
21             super(firstName, lastName); // Call parent constructor
22             this.matriculationNumber = nextMatriculationNumber++;
```

Java

```
23     }
24
25     public int getMatriculationNumber() {
26         return matriculationNumber;
27     }
28 }
```

Using constants for scalability:

```
1     public class StudentManager {
2         private static final int MAX_ANZAHL = 500;
3         private Student[] students = new Student[MAX_ANZAHL];
4
5         // Menu options based on MAX_ANZAHL
6         if (choice == MAX_ANZAHL + 1) {
7             displayAllStudents();
8         } else if (choice == MAX_ANZAHL + 2) {
9             System.out.println("Maximum students: " + MAX_ANZAHL);
10        }
11    }
```

Java

Solution

Person.java:

```
1     public class Person {
2         protected String firstName;
3         protected String lastName;
4         protected int day;
5         protected int month;
6         protected int year;
7         private static int personCount = 0;
8
9         // Constructor
10        public Person(String firstName, String lastName, int day, int month, int
year) {
11            this.firstName = firstName;
12            this.lastName = lastName;
13            this.day = day;
14            this.month = month;
15            this.year = year;
16            personCount++;
17        }
18
19        // Getter methods
20        public String getFirstName() {
21            return firstName;
22        }
23
24        public String getLastName() {
```

Java

```
25     return lastName;
26 }
27
28 public int getDay() {
29     return day;
30 }
31
32 public int getMonth() {
33     return month;
34 }
35
36 public int getYear() {
37     return year;
38 }
39
40 // Setter methods
41 public void setFirstName(String firstName) {
42     this.firstName = firstName;
43 }
44
45 public void setLastName(String lastName) {
46     this.lastName = lastName;
47 }
48
49 public void setDay(int day) {
50     this.day = day;
51 }
52
53 public void setMonth(int month) {
54     if (month ≥ 1 && month ≤ 12) {
55         this.month = month;
56     } else {
57         System.out.println("Invalid month! Must be between 1-12.");
58     }
59 }
60
61 public void setYear(int year) {
62     this.year = year;
63 }
64
65 public static int getPersonCount() {
66     return personCount;
67 }
68
69 // Display person information
70 public void displayInfo() {
```

```
71     System.out.println("Name: " + firstName + " " + lastName);
72     System.out.println("Date: " + day + "." + month + "." + year);
73 }
74
75     // Check if three years have passed
76     public boolean canChangeName(int newDay, int newMonth, int newYear) {
77         int yearsDiff = newYear - this.year;
78         if (yearsDiff < 3) return false;
79         if (yearsDiff == 3) {
80             if (newMonth < this.month) return false;
81             if (newMonth == this.month && newDay < this.day) return false;
82         }
83         return true;
84     }
85 }
```

Student.java:

```
1     public class Student extends Person {
2         private int matriculationNumber;
3         private static int nextMatriculationNumber = 1001;
4
5         // Constructor
6         public Student(String firstName, String lastName, int day, int month, int
year) {
7             super(firstName, lastName, day, month, year);
8             this.matriculationNumber = nextMatriculationNumber++;
9         }
10
11         // Getter for matriculation number
12         public int getMatriculationNumber() {
13             return matriculationNumber;
14         }
15
16         // Override displayInfo to include matriculation number
17         @Override
18         public void displayInfo() {
19             System.out.println("Matriculation Number: " + matriculationNumber);
20             super.displayInfo();
21         }
22     }
```

Java

StudentManagementSystem.java:

```
1     import java.util.Scanner;
2
3     public class StudentManagementSystem {
4         private static final int MAX_ANZAHL = 500;
5     }
```

Java

```
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8
9         // Create array of Student objects
10        Student[] students = new Student[MAX_ANZAHL];
11        int studentCount = 0;
12
13        // Initialize with some students
14        students[studentCount++] = new Student("Max", "Mustermann", 1, 1, 2020);
15        students[studentCount++] = new Student("Anna", "Schmidt", 15, 6, 2019);
16        students[studentCount++] = new Student("Peter", "Mueller", 20, 12, 2021);
17
18        boolean running = true;
19
20        while (running) {
21            System.out.println("\n≡ Student Management System ≡");
22            System.out.println("Enter student number (1-" + studentCount + ") to modify");
23            System.out.println("Enter " + (MAX_ANZAHL + 1) + " to display all students");
24            System.out.println("Enter " + (MAX_ANZAHL + 2) + " to display maximum capacity");
25            System.out.println("Enter 0 to exit");
26            System.out.print("Your choice: ");
27
28            int choice = scanner.nextInt();
29            scanner.nextLine();
30
31            if (choice == 0) {
32                System.out.println("Program terminated.");
33                running = false;
34            } else if (choice == MAX_ANZAHL + 1) {
35                // Display all students
36                System.out.println("\n≡ All Students ≡");
37                for (int i = 0; i < studentCount; i++) {
38                    System.out.println("\nStudent " + (i + 1) + ":");
39                    students[i].displayInfo();
40                }
41            } else if (choice == MAX_ANZAHL + 2) {
42                // Display maximum capacity
43                System.out.println("\nMaximum students manageable: " + MAX_ANZAHL);
44                System.out.println("Current students: " + studentCount);
45            } else if (choice ≥ 1 && choice ≤ studentCount) {
46                // Modify selected student
47                Student selectedStudent = students[choice - 1];
48
49                System.out.print("Enter new first name: ");
```

```
50         String newFirst = scanner.nextLine();
51         selectedStudent.setFirstName(newFirst);
52
53         System.out.print("Enter new last name: ");
54         String newLast = scanner.nextLine();
55         selectedStudent.setLastName(newLast);
56
57         System.out.print("Enter day: ");
58         int newDay = scanner.nextInt();
59         selectedStudent.setDay(newDay);
60
61         System.out.print("Enter month (1-12): ");
62         int newMonth = scanner.nextInt();
63         selectedStudent.setMonth(newMonth);
64
65         System.out.print("Enter year: ");
66         int newYear = scanner.nextInt();
67         scanner.nextLine();
68
69         if (selectedStudent.canChangeName(newDay, newMonth, newYear)) {
70             selectedStudent.setYear(newYear);
71             System.out.println("\nUpdate successful!");
72             selectedStudent.displayInfo();
73         } else {
74             System.out.println("\nError: Three years have not passed!");
75         }
76     } else {
77         System.out.println("Invalid choice!");
78     }
79 }
80
81 scanner.close();
82 }
83 }
```

Task 2: Vehicle Hierarchy with Polymorphism

Create a vehicle management system that demonstrates inheritance and polymorphism. This task will help you understand method overriding and polymorphic behavior.

Create a class hierarchy for different types of vehicles:

- Base class `Vehicle` with common properties (brand, model, year)
- Subclasses `Car`, `Motorcycle`, and `Truck` that extend `Vehicle`
- Each subclass should have specific attributes and override methods appropriately

Your program should:

1. Create a base `Vehicle` class with:
 - Protected attributes: brand, model, year
 - Constructor and getter methods
 - A `displayInfo()` method that shows basic vehicle information
 - An abstract or overridable `calculateMaintenanceCost()` method
2. Create subclasses that:
 - Add specific attributes (e.g., `numberOfDoors` for `Car`, `cargoCapacity` for `Truck`)
 - Override `displayInfo()` to include subclass-specific information
 - Implement `calculateMaintenanceCost()` with different formulas
3. Create an array of `Vehicle` objects containing different types
4. Use polymorphism to call methods on all vehicles in the array

Requirements

- Use inheritance with `extends` keyword
- Demonstrate method overriding with `@Override` annotation
- Create polymorphic behavior using `Vehicle` array containing different subclasses
- Use protected access modifiers appropriately

Note about `@Override`: The `@Override` annotation is a helpful tool that tells Java you intend to override a method from the parent class. While not strictly required, it's considered good practice because it helps catch errors at compile time. If you misspell a method name or get the parameters wrong, Java will give you an error instead of accidentally creating a new method.

Assistance

Basic vehicle hierarchy:

```
1      public abstract class Vehicle {
2          protected String brand;
3          protected String model;
4          protected int year;
5
6          public Vehicle(String brand, String model, int year) {
7              this.brand = brand;
8              this.model = model;
9              this.year = year;
10         }
11
12         public void displayInfo() {
13             System.out.println(year + " " + brand + " " + model);
14         }
15
16         public abstract double calculateMaintenanceCost();
17     }
18
19     public class Car extends Vehicle {
20         private int numberOfDoors;
```

Java

```
21
22     public Car(String brand, String model, int year, int doors) {
23         super(brand, model, year);
24         this.numberOfDoors = doors;
25     }
26
27     @Override
28     public void displayInfo() {
29         super.displayInfo();
30         System.out.println("Doors: " + numberOfDoors);
31     }
32
33     @Override
34     public double calculateMaintenanceCost() {
35         return (2025 - year) * 150.0; // Older cars cost more
36     }
37 }
```

Polymorphic array usage:

```
1     Vehicle[] vehicles = {
2         new Car("Toyota", "Camry", 2020, 4),
3         new Motorcycle("Honda", "CBR", 2021, 600),
4         new Truck("Ford", "F-150", 2019, 1500)
5     };
6
7     for (Vehicle v : vehicles) {
8         v.displayInfo(); // Calls overridden method
9         System.out.println("Maintenance: $" + v.calculateMaintenanceCost());
10    }
```

Java

Solution

Vehicle.java (Abstract Base Class):

```
1     public abstract class Vehicle {
2         protected String brand;
3         protected String model;
4         protected int year;
5
6         // Constructor
7         public Vehicle(String brand, String model, int year) {
8             this.brand = brand;
9             this.model = model;
10            this.year = year;
11        }
12
13        // Getters
14        public String getBrand() {
15            return brand;
```

Java

```
16     }
17
18     public String getModel() {
19         return model;
20     }
21
22     public int getYear() {
23         return year;
24     }
25
26     // Display basic vehicle information
27     public void displayInfo() {
28         System.out.println(year + " " + brand + " " + model);
29     }
30
31     // Abstract method - must be implemented by subclasses
32     public abstract double calculateMaintenanceCost();
33 }
```

Car.java:

```
1     public class Car extends Vehicle {
2         private int numberOfDoors;
3
4         // Constructor
5         public Car(String brand, String model, int year, int numberOfDoors) {
6             super(brand, model, year);
7             this.numberOfDoors = numberOfDoors;
8         }
9
10        public int getNumberOfDoors() {
11            return numberOfDoors;
12        }
13
14        // Override displayInfo to include car-specific information
15        @Override
16        public void displayInfo() {
17            super.displayInfo();
18            System.out.println("Type: Car");
19            System.out.println("Number of doors: " + numberOfDoors);
20        }
21
22        // Implement abstract method
23        @Override
24        public double calculateMaintenanceCost() {
25            // Cars: base cost + age factor
26            int age = 2025 - year;
27            return 200.0 + (age * 50.0);
```

Java

```
28     }
29 }
```

Motorcycle.java:

```
1  public class Motorcycle extends Vehicle {
2      private int engineCapacity; // in cc
3
4      // Constructor
5      public Motorcycle(String brand, String model, int year, int engineCapacity)
6      {
7          super(brand, model, year);
8          this.engineCapacity = engineCapacity;
9      }
10
11     public int getEngineCapacity() {
12         return engineCapacity;
13     }
14
15     @Override
16     public void displayInfo() {
17         super.displayInfo();
18         System.out.println("Type: Motorcycle");
19         System.out.println("Engine capacity: " + engineCapacity + "cc");
20     }
21
22     @Override
23     public double calculateMaintenanceCost() {
24         // Motorcycles: lower base cost but engine size affects maintenance
25         int age = 2025 - year;
26         return 100.0 + (age * 30.0) + (engineCapacity / 100.0);
27     }
28 }
```

Java

Truck.java:

```
1  public class Truck extends Vehicle {
2      private int cargoCapacity; // in kg
3
4      // Constructor
5      public Truck(String brand, String model, int year, int cargoCapacity) {
6          super(brand, model, year);
7          this.cargoCapacity = cargoCapacity;
8      }
9
10     public int getCargoCapacity() {
11         return cargoCapacity;
12     }
13 }
```

Java

```

14     @Override
15     public void displayInfo() {
16         super.displayInfo();
17         System.out.println("Type: Truck");
18         System.out.println("Cargo capacity: " + cargoCapacity + "kg");
19     }
20
21     @Override
22     public double calculateMaintenanceCost() {
23         // Trucks: higher maintenance due to heavy use
24         int age = 2025 - year;
25         return 400.0 + (age * 80.0) + (cargoCapacity / 10.0);
26     }
27 }

```

VehicleManagementDemo.java:

```

1     public class VehicleManagementDemo {
2         public static void main(String[] args) {
3             // Create polymorphic array of vehicles
4             Vehicle[] vehicles = new Vehicle[5];
5             vehicles[0] = new Car("Toyota", "Camry", 2020, 4);
6             vehicles[1] = new Motorcycle("Honda", "CBR600RR", 2021, 600);
7             vehicles[2] = new Truck("Ford", "F-150", 2019, 1500);
8             vehicles[3] = new Car("BMW", "X5", 2022, 5);
9             vehicles[4] = new Motorcycle("Yamaha", "R1", 2018, 1000);
10
11             System.out.println("≡ Vehicle Fleet Management ≡\n");
12
13             // Demonstrate polymorphism
14             double totalMaintenanceCost = 0;
15
16             for (int i = 0; i < vehicles.length; i++) {
17                 System.out.println("Vehicle " + (i + 1) + ":");
18                 vehicles[i].displayInfo(); // Polymorphic call
19                 double cost = vehicles[i].calculateMaintenanceCost(); // Polymorphic
20                                     call
21                 System.out.printf("Annual Maintenance Cost: $%.2f\n", cost);
22                 totalMaintenanceCost += cost;
23                 System.out.println();
24             }
25
26             System.out.printf("≡ Total Fleet Maintenance Cost: $%.2f ≡\n",
27                               totalMaintenanceCost);
28         }
29     }

```

Java

Task 3: Animal Classification System

Create an animal classification system that demonstrates advanced inheritance concepts including abstract classes and interfaces. This task focuses on designing hierarchical relationships.

Design a system that models different types of animals with their behaviors:

- Abstract base class `Animal` with common properties
- Interface `Flyable` for animals that can fly
- Interface `Swimmable` for animals that can swim
- Concrete animal classes that implement appropriate interfaces

Your program should:

1. Create an abstract `Animal` class with:
 - Protected attributes: `name`, `species`, `age`
 - Abstract method `makeSound()`
 - Concrete method `displayInfo()`
2. Create interfaces:
 - `Flyable` with method `fly()`
 - `Swimmable` with method `swim()`
3. Create concrete animal classes:
 - `Bird` (extends `Animal`, implements `Flyable`)
 - `Fish` (extends `Animal`, implements `Swimmable`)
 - `Duck` (extends `Animal`, implements both `Flyable` and `Swimmable`)
 - `Dog` (extends `Animal`, no additional interfaces)
4. Demonstrate multiple inheritance through interfaces

Requirements

- Use abstract classes and methods
- Implement multiple interfaces in same class
- Override abstract methods in concrete classes
- Demonstrate interface polymorphism

Assistance

Abstract class and interface structure:

```
1      public abstract class Animal {
2          protected String name;
3          protected String species;
4          protected int age;
5
6          public Animal(String name, String species, int age) {
7              this.name = name;
8              this.species = species;
9              this.age = age;
10         }
11
12         public void displayInfo() {
13             System.out.println(name + " is a " + age + " year old " + species);
14         }
15
16         public abstract void makeSound();
17     }
18
19     public interface Flyable {
20         void fly();
21         default int getMaxAltitude() { return 1000; } // Default method
```

Java

```
22     }
23
24     public interface Swimmable {
25         void swim();
26     }
27
28     public class Duck extends Animal implements Flyable, Swimmable {
29         public Duck(String name, int age) {
30             super(name, "Duck", age);
31         }
32
33         @Override
34         public void makeSound() {
35             System.out.println("Quack!");
36         }
37
38         @Override
39         public void fly() {
40             System.out.println(name + " is flying over the pond!");
41         }
42
43         @Override
44         public void swim() {
45             System.out.println(name + " is swimming in the water!");
46         }
47     }
```

Solution

Animal.java (Abstract Base Class):

```
1     public abstract class Animal {
2         protected String name;
3         protected String species;
4         protected int age;
5
6         // Constructor
7         public Animal(String name, String species, int age) {
8             this.name = name;
9             this.species = species;
10            this.age = age;
11        }
12
13        // Concrete method
14        public void displayInfo() {
15            System.out.println(name + " is a " + age + " year old " + species);
16        }
17    }
```

Java

```
18     // Getters
19     public String getName() {
20         return name;
21     }
22
23     public String getSpecies() {
24         return species;
25     }
26
27     public int getAge() {
28         return age;
29     }
30
31     // Abstract method - must be implemented by subclasses
32     public abstract void makeSound();
33 }
```

Flyable.java (Interface):

```
1     public interface Flyable {
2         void fly();
3
4         // Default method with default implementation
5         default int getMaxAltitude() {
6             return 1000; // Default altitude in meters
7         }
8     }
```

Java

Swimmable.java (Interface):

```
1     public interface Swimmable {
2         void swim();
3
4         // Default method
5         default int getMaxDepth() {
6             return 10; // Default depth in meters
7         }
8     }
```

Java

Bird.java:

```
1     public class Bird extends Animal implements Flyable {
2         private String wingSpan;
3
4         public Bird(String name, int age, String wingSpan) {
5             super(name, "Bird", age);
6             this.wingSpan = wingSpan;
7         }
8
9         @Override
```

Java

```
10     public void makeSound() {
11         System.out.println(name + " says: Tweet tweet!");
12     }
13
14     @Override
15     public void fly() {
16         System.out.println(name + " is flying through the air!");
17     }
18
19     @Override
20     public int getMaxAltitude() {
21         return 3000; // Birds can fly higher
22     }
23
24     public String getWingSpan() {
25         return wingSpan;
26     }
27 }
```

Fish.java:

```
1     public class Fish extends Animal implements Swimmable {
2         private String waterType;
3
4         public Fish(String name, int age, String waterType) {
5             super(name, "Fish", age);
6             this.waterType = waterType;
7         }
8
9         @Override
10        public void makeSound() {
11            System.out.println(name + " makes bubble sounds...");
12        }
13
14        @Override
15        public void swim() {
16            System.out.println(name + " is swimming in " + waterType + " water!");
17        }
18
19        @Override
20        public int getMaxDepth() {
21            return 100; // Fish can dive deeper
22        }
23
24        public String getWaterType() {
25            return waterType;
26        }
27    }
```

Java

Duck.java (Multiple Interfaces):

```
1    public class Duck extends Animal implements Flyable, Swimmable {
2
3        public Duck(String name, int age) {
4            super(name, "Duck", age);
5        }
6
7        @Override
8        public void makeSound() {
9            System.out.println(name + " says: Quack quack!");
10       }
11
12       @Override
13       public void fly() {
14           System.out.println(name + " is flying over the pond!");
15       }
16
17       @Override
18       public void swim() {
19           System.out.println(name + " is swimming gracefully!");
20       }
21
22       @Override
23       public int getMaxAltitude() {
24           return 1500; // Ducks fly at moderate altitude
25       }
26
27       @Override
28       public int getMaxDepth() {
29           return 5; // Ducks dive shallow
30       }
31   }
```

Java

Dog.java:

```
1    public class Dog extends Animal {
2        private String breed;
3
4        public Dog(String name, int age, String breed) {
5            super(name, "Dog", age);
6            this.breed = breed;
7        }
8
9        @Override
10       public void makeSound() {
11           System.out.println(name + " says: Woof woof!");
12       }
13   }
```

Java

```
14     public String getBreed() {
15         return breed;
16     }
17
18     public void fetch() {
19         System.out.println(name + " is fetching the ball!");
20     }
21 }
```

AnimalClassificationDemo.java:

```
1     public class AnimalClassificationDemo {
2         public static void main(String[] args) {
3             // Create various animals
4             Animal[] animals = new Animal[5];
5             animals[0] = new Bird("Eagle", 5, "2 meters");
6             animals[1] = new Fish("Salmon", 3, "fresh");
7             animals[2] = new Duck("Donald", 2);
8             animals[3] = new Dog("Buddy", 4, "Golden Retriever");
9             animals[4] = new Bird("Sparrow", 1, "20 cm");
10
11             System.out.println("≡≡≡ Animal Classification System ≡≡≡\n");
12
13             // Demonstrate polymorphism
14             for (Animal animal : animals) {
15                 animal.displayInfo();
16                 animal.makeSound();
17                 System.out.println();
18             }
19
20             // Demonstrate interface polymorphism
21             System.out.println("≡≡≡ Flying Animals ≡≡≡");
22             for (Animal animal : animals) {
23                 if (animal instanceof Flyable) {
24                     Flyable flyer = (Flyable) animal;
25                     flyer.fly();
26                     System.out.println("Max altitude: " + flyer.getMaxAltitude() + "
27                                     meters");
28                 }
29
30             System.out.println("\n≡≡≡ Swimming Animals ≡≡≡");
31             for (Animal animal : animals) {
32                 if (animal instanceof Swimmable) {
33                     Swimmable swimmer = (Swimmable) animal;
34                     swimmer.swim();
35                     System.out.println("Max depth: " + swimmer.getMaxDepth() + "
36                                     meters");
37                 }
38             }
39         }
40     }
```

Java

```
37     }
38
39     // Duck can both fly and swim
40     System.out.println("\n=== Special: Duck can do both! ===");
41     Duck duck = (Duck) animals[2];
42     duck.fly();
43     duck.swim();
44 }
45 }
```

Task 4: Collections in Java Programming

Enhance your student management system by implementing generic collections and data structures. This task introduces Java Collections Framework, generics, and advanced data manipulation techniques.

Create a comprehensive student management system using Java collections:

- Replace arrays with appropriate Collection class called ArrayList
- Add search and sorting functionality
- Create custom comparators for different sorting criteria

Your program should:

1. Convert your Student array to use ArrayList<Student>
2. Implement a StudentRegistry class that uses:
 - ArrayList<Student> for maintaining insertion order
4. Implement multiple sorting options with your favorite sorting algorithm (refer to the last lab):
 - Sort by name (alphabetical)
 - Sort by matriculation number
 - Sort by age (if birth date is available)
5. Add advanced search functionality with your favorite search algorithm:
 - Find students by partial name match
 - Find students by matriculation number range
 - Find students older/younger than specific age

Requirements

- Use appropriate Collection interfaces and implementations
- Implement generic methods with proper type parameters
- Use lambda expressions for predicates and comparators
- Demonstrate the difference between List, Set, and Map usage
- Handle duplicate prevention using Set characteristics

Assistance

Collection usage patterns:

```
1 public class StudentRegistry {
2     private List<Student> students = new ArrayList<>();
3
4     public void addStudent(Student student) {
5         students.add(student);
6     }
7 }
```

Java

Solution

Student.java (Updated for Collections):

```
1 public class Student {
2     private String firstName;
3     private String lastName;
4     private int matriculationNumber;
5     private int age;
6     private static int nextMatriculationNumber = 1001;
7
8     public Student(String firstName, String lastName, int age) {
9         this.firstName = firstName;
10        this.lastName = lastName;
11        this.age = age;
12        this.matriculationNumber = nextMatriculationNumber++;
13    }
```

Java

```
14
15     public String getFirstName() {
16         return firstName;
17     }
18
19     public String getLastName() {
20         return lastName;
21     }
22
23     public int getMatriculationNumber() {
24         return matriculationNumber;
25     }
26
27     public int getAge() {
28         return age;
29     }
30
31     public String getFullName() {
32         return firstName + " " + lastName;
33     }
34
35     @Override
36     public String toString() {
37         return "Student{" +
38             "matricNo=" + matriculationNumber +
39             ", name='" + getFullName() + '\'' +
40             ", age=" + age +
41             '}';
42     }
43 }
```

StudentRegistry.java:

```
1     import java.util.ArrayList;
2     import java.util.List;
3
4     public class StudentRegistry {
5         private List<Student> students;
6
7         public StudentRegistry() {
8             this.students = new ArrayList<>();
9         }
10
11         // Add student
12         public void addStudent(Student student) {
13             students.add(student);
14             System.out.println("Student added: " + student.getFullName());
15         }
```

Java

```
16
17     // Display all students
18     public void displayAllStudents() {
19         if (students.isEmpty()) {
20             System.out.println("No students in registry.");
21             return;
22         }
23
24         System.out.println("\n≡≡≡ All Students ≡≡≡");
25         for (Student student : students) {
26             System.out.println(student);
27         }
28     }
29
30     // Sort by name (Bubble Sort)
31     public void sortByName() {
32         for (int i = 0; i < students.size() - 1; i++) {
33             for (int j = 0; j < students.size() - i - 1; j++) {
34                 if (students.get(j).getFullName()
35                     .compareTo(students.get(j + 1).getFullName()) > 0) {
36                     // Swap
37                     Student temp = students.get(j);
38                     students.set(j, students.get(j + 1));
39                     students.set(j + 1, temp);
40                 }
41             }
42         }
43         System.out.println("Students sorted by name.");
44     }
45
46     // Sort by matriculation number (Bubble Sort)
47     public void sortByMatriculationNumber() {
48         for (int i = 0; i < students.size() - 1; i++) {
49             for (int j = 0; j < students.size() - i - 1; j++) {
50                 if (students.get(j).getMatriculationNumber() >
51                     students.get(j + 1).getMatriculationNumber()) {
52                     // Swap
53                     Student temp = students.get(j);
54                     students.set(j, students.get(j + 1));
55                     students.set(j + 1, temp);
56                 }
57             }
58         }
59         System.out.println("Students sorted by matriculation number.");
60     }
61 }
```

```
62      // Sort by age (Bubble Sort)
63      public void sortByAge() {
64          for (int i = 0; i < students.size() - 1; i++) {
65              for (int j = 0; j < students.size() - i - 1; j++) {
66                  if (students.get(j).getAge() > students.get(j + 1).getAge()) {
67                      // Swap
68                      Student temp = students.get(j);
69                      students.set(j, students.get(j + 1));
70                      students.set(j + 1, temp);
71                  }
72              }
73          }
74          System.out.println("Students sorted by age.");
75      }
76
77      // Search by partial name match (Linear Search)
78      public List<Student> searchByPartialName(String partialName) {
79          List<Student> results = new ArrayList<>();
80          String searchTerm = partialName.toLowerCase();
81
82          for (Student student : students) {
83              if (student.getFullName().toLowerCase().contains(searchTerm)) {
84                  results.add(student);
85              }
86          }
87          return results;
88      }
89
90      // Search by matriculation number (Linear Search)
91      public Student searchByMatriculationNumber(int matricNo) {
92          for (Student student : students) {
93              if (student.getMatriculationNumber() == matricNo) {
94                  return student;
95              }
96          }
97          return null;
98      }
99
100     // Find students in matriculation number range
101     public List<Student> findByMatriculationRange(int min, int max) {
102         List<Student> results = new ArrayList<>();
103         for (Student student : students) {
104             int matricNo = student.getMatriculationNumber();
105             if (matricNo ≥ min && matricNo ≤ max) {
106                 results.add(student);
107             }
108         }
109     }
```

```
108     }
109     return results;
110 }
111
112 // Find students older than age
113 public List<Student> findOlderThan(int age) {
114     List<Student> results = new ArrayList<>();
115     for (Student student : students) {
116         if (student.getAge() > age) {
117             results.add(student);
118         }
119     }
120     return results;
121 }
122
123 // Find students younger than age
124 public List<Student> findYoungerThan(int age) {
125     List<Student> results = new ArrayList<>();
126     for (Student student : students) {
127         if (student.getAge() < age) {
128             results.add(student);
129         }
130     }
131     return results;
132 }
133
134 public int getStudentCount() {
135     return students.size();
136 }
137 }
```

CollectionsDemo.java:

```
1  import java.util.List;
2  import java.util.Scanner;
3
4  public class CollectionsDemo {
5      public static void main(String[] args) {
6          Scanner scanner = new Scanner(System.in);
7          StudentRegistry registry = new StudentRegistry();
8
9          // Add sample students
10         registry.addStudent(new Student("Alice", "Johnson", 22));
11         registry.addStudent(new Student("Bob", "Smith", 20));
12         registry.addStudent(new Student("Charlie", "Brown", 23));
13         registry.addStudent(new Student("Diana", "Prince", 21));
14         registry.addStudent(new Student("Eve", "Anderson", 19));
15     }
```

[Java](#)

```
16     boolean running = true;
17
18     while (running) {
19         System.out.println("\n≡≡≡ Student Registry with Collections ≡≡≡");
20         System.out.println("1: Add student");
21         System.out.println("2: Display all students");
22         System.out.println("3: Sort by name");
23         System.out.println("4: Sort by matriculation number");
24         System.out.println("5: Sort by age");
25         System.out.println("6: Search by name");
26         System.out.println("7: Search by matriculation number");
27         System.out.println("8: Find by matriculation range");
28         System.out.println("9: Find by age filter");
29         System.out.println("0: Exit");
30         System.out.print("Your choice: ");
31
32         int choice = scanner.nextInt();
33         scanner.nextLine();
34
35         switch (choice) {
36             case 0:
37                 running = false;
38                 System.out.println("Exiting...");
39                 break;
40
41             case 1:
42                 System.out.print("First name: ");
43                 String firstName = scanner.nextLine();
44                 System.out.print("Last name: ");
45                 String lastName = scanner.nextLine();
46                 System.out.print("Age: ");
47                 int age = scanner.nextInt();
48                 registry.addStudent(new Student(firstName, lastName, age));
49                 break;
50
51             case 2:
52                 registry.displayAllStudents();
53                 break;
54
55             case 3:
56                 registry.sortByName();
57                 registry.displayAllStudents();
58                 break;
59
60             case 4:
61                 registry.sortByMatriculationNumber();
```

```
62         registry.displayAllStudents();
63         break;
64
65     case 5:
66         registry.sortByAge();
67         registry.displayAllStudents();
68         break;
69
70     case 6:
71         System.out.print("Enter name to search: ");
72         String searchName = scanner.nextLine();
73         List<Student> nameResults =
74             registry.searchByPartialName(searchName);
75         System.out.println("Found " + nameResults.size() + " students:");
76         for (Student s : nameResults) {
77             System.out.println(s);
78         }
79         break;
80
81     case 7:
82         System.out.print("Enter matriculation number: ");
83         int matricNo = scanner.nextInt();
84         Student found = registry.searchByMatriculationNumber(matricNo);
85         if (found != null) {
86             System.out.println("Found: " + found);
87         } else {
88             System.out.println("Student not found.");
89         }
90         break;
91
92     case 8:
93         System.out.print("Enter minimum matriculation number: ");
94         int min = scanner.nextInt();
95         System.out.print("Enter maximum matriculation number: ");
96         int max = scanner.nextInt();
97         List<Student> rangeResults = registry.findByMatriculationRange(min,
98             max);
99         System.out.println("Found " + rangeResults.size() + " students:");
100        for (Student s : rangeResults) {
101            System.out.println(s);
102        }
103        break;
104
105     case 9:
106         System.out.println("1: Older than");
107         System.out.println("2: Younger than");
108         System.out.print("Choice: ");
```

```
107         int ageChoice = scanner.nextInt();
108         System.out.print("Enter age: ");
109         int ageFilter = scanner.nextInt();
110
111         List<Student> ageResults;
112         if (ageChoice == 1) {
113             ageResults = registry.findOlderThan(ageFilter);
114             System.out.println("Students older than " + ageFilter + ":");
115         } else {
116             ageResults = registry.findYoungerThan(ageFilter);
117             System.out.println("Students younger than " + ageFilter + ":");
118         }
119         for (Student s : ageResults) {
120             System.out.println(s);
121         }
122         break;
123
124         default:
125             System.out.println("Invalid choice!");
126     }
127 }
128
129 scanner.close();
130 }
131 }
```

Task 5: Design Patterns Implementation

Implement common design patterns to improve your student management system's architecture. This task introduces Singleton, Observer, and Factory patterns in a practical context.

Apply design patterns to create a robust and maintainable system:

- Observer pattern for real-time notifications
- Factory pattern for creating different types of academic entities
- Strategy pattern for different grading systems

Your program should:

1. Create Observer pattern for student events:
 - StudentObserver interface for notifications
 - StudentSubject for managing observers
 - Notify observers when students are added, removed, or modified
2. Use Factory pattern for academic entities:
 - PersonFactory that creates different types (Student, Professor, Staff)
 - CourseFactory for different course types (Lecture, Lab, Seminar)
3. Implement Strategy pattern for grading:
 - Different grading strategies (German grades, ECTS, Pass/Fail)
 - Allow runtime switching between grading systems

Requirements

- Implement at least three different design patterns
- Demonstrate loose coupling between components
- Use interfaces to define contracts between classes
- Show how patterns improve code maintainability and extensibility

Assistance

Observer pattern structure:

```
1      public interface StudentObserver {
2          void onStudentAdded(Student student);
3          void onStudentRemoved(Student student);
4          void onStudentModified(Student student);
5      }
6
7      public class StudentRegistry {
8          private List<StudentObserver> observers = new ArrayList<>();
9          private List<Student> students = new ArrayList<>();
10
11         public void addObserver(StudentObserver observer) {
12             observers.add(observer);
13         }
14
15         public void addStudent(Student student) {
16             students.add(student);
17             notifyObservers(observer → observer.onStudentAdded(student));
18         }
19
20         private void notifyObservers(Consumer<StudentObserver> action) {
21             observers.forEach(action);
22         }
23     }
```

Java

Factory pattern example:

```
1 public abstract class PersonFactory {
2     public static Person createPerson(String type, String firstName, String
        lastName) {
3         switch (type.toLowerCase()) {
4             case "student":
5                 return new Student(firstName, lastName);
6             case "professor":
7                 return new Professor(firstName, lastName);
8             case "staff":
9                 return new Staff(firstName, lastName);
10            default:
11                throw new IllegalArgumentException("Unknown person type: " +
                    type);
12        }
13    }
14 }
```

Java

Solution

StudentObserver.java (Observer Pattern - Interface):

```
1 public interface StudentObserver {
2     void onStudentAdded(Student student);
3     void onStudentRemoved(Student student);
4     void onStudentModified(Student student);
5 }
```

Java

ConsoleLogger.java (Observer Pattern - Concrete Observer):

```
1 public class ConsoleLogger implements StudentObserver {
2     @Override
3     public void onStudentAdded(Student student) {
4         System.out.println("[LOG] Student added: " + student.getFullName());
5     }
6
7     @Override
8     public void onStudentRemoved(Student student) {
9         System.out.println("[LOG] Student removed: " + student.getFullName());
10    }
11
12    @Override
13    public void onStudentModified(Student student) {
14        System.out.println("[LOG] Student modified: " + student.getFullName());
15    }
16 }
```

Java

ObservableStudentRegistry.java (Observer Pattern - Subject):

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class ObservableStudentRegistry {
```

Java

```
5     private List<Student> students;
6     private List<StudentObserver> observers;
7
8     public ObservableStudentRegistry() {
9         this.students = new ArrayList<>();
10        this.observers = new ArrayList<>();
11    }
12
13    // Register observer
14    public void addObserver(StudentObserver observer) {
15        observers.add(observer);
16    }
17
18    // Remove observer
19    public void removeObserver(StudentObserver observer) {
20        observers.remove(observer);
21    }
22
23    // Notify all observers
24    private void notifyStudentAdded(Student student) {
25        for (StudentObserver observer : observers) {
26            observer.onStudentAdded(student);
27        }
28    }
29
30    private void notifyStudentRemoved(Student student) {
31        for (StudentObserver observer : observers) {
32            observer.onStudentRemoved(student);
33        }
34    }
35
36    private void notifyStudentModified(Student student) {
37        for (StudentObserver observer : observers) {
38            observer.onStudentModified(student);
39        }
40    }
41
42    // Student operations
43    public void addStudent(Student student) {
44        students.add(student);
45        notifyStudentAdded(student);
46    }
47
48    public void removeStudent(Student student) {
49        if (students.remove(student)) {
50            notifyStudentRemoved(student);
```

```

51     }
52     }
53
54     public void modifyStudent(Student student) {
55         notifyStudentModified(student);
56     }
57
58     public List<Student> getStudents() {
59         return new ArrayList<>(students);
60     }
61     }

```

PersonFactory.java (Factory Pattern):

```

1     public class PersonFactory {
2         public static Person createPerson(String type, String firstName, String
3             lastName, int age) {
4             switch (type.toLowerCase()) {
5                 case "student":
6                     return new Student(firstName, lastName, age);
7                 case "professor":
8                     return new Professor(firstName, lastName, age);
9                 case "staff":
10                     return new Staff(firstName, lastName, age);
11                 default:
12                     throw new IllegalArgumentException("Unknown person type: " + type);
13             }
14         }
15     }

```

Java

Professor.java (For Factory Pattern):

```

1     public class Professor extends Person {
2         private String department;
3
4         public Professor(String firstName, String lastName, int age) {
5             super(firstName, lastName, 1, 1, 2020);
6             this.department = "Computer Science";
7         }
8
9         public String getDepartment() {
10             return department;
11         }
12
13         public void setDepartment(String department) {
14             this.department = department;
15         }
16
17         @Override

```

Java

```
18     public void displayInfo() {
19         System.out.println("Professor: " + getFirstName() + " " + getLastName());
20         System.out.println("Department: " + department);
21     }
22 }
```

Staff.java (For Factory Pattern):

```
1     public class Staff extends Person {
2         private String role;
3
4         public Staff(String firstName, String lastName, int age) {
5             super(firstName, lastName, 1, 1, 2020);
6             this.role = "Administrative Staff";
7         }
8
9         public String getRole() {
10             return role;
11         }
12
13         public void setRole(String role) {
14             this.role = role;
15         }
16
17         @Override
18         public void displayInfo() {
19             System.out.println("Staff: " + getFirstName() + " " + getLastName());
20             System.out.println("Role: " + role);
21         }
22     }
```

Java

GradingStrategy.java (Strategy Pattern - Interface):

```
1     public interface GradingStrategy {
2         String calculateGrade(double score);
3         String getGradingSystemName();
4     }
```

Java

GermanGradingStrategy.java:

```
1     public class GermanGradingStrategy implements GradingStrategy {
2         @Override
3         public String calculateGrade(double score) {
4             if (score ≥ 90) return "1.0 (Sehr gut)";
5             else if (score ≥ 80) return "2.0 (Gut)";
6             else if (score ≥ 70) return "3.0 (Befriedigend)";
7             else if (score ≥ 60) return "4.0 (Ausreichend)";
8             else return "5.0 (Nicht bestanden)";
9         }
10     }
```

Java

```
11     @Override
12     public String getGradingSystemName() {
13         return "German Grading System";
14     }
15 }
```

ECTSGradingStrategy.java:

```
1     public class ECTSGradingStrategy implements GradingStrategy {
2         @Override
3         public String calculateGrade(double score) {
4             if (score ≥ 90) return "A (Excellent)";
5             else if (score ≥ 80) return "B (Very Good)";
6             else if (score ≥ 70) return "C (Good)";
7             else if (score ≥ 60) return "D (Satisfactory)";
8             else if (score ≥ 50) return "E (Sufficient)";
9             else return "F (Fail)";
10        }
11
12        @Override
13        public String getGradingSystemName() {
14            return "ECTS Grading System";
15        }
16    }
```

Java

PassFailGradingStrategy.java:

```
1     public class PassFailGradingStrategy implements GradingStrategy {
2         @Override
3         public String calculateGrade(double score) {
4             return score ≥ 60 ? "Pass" : "Fail";
5         }
6
7         @Override
8         public String getGradingSystemName() {
9             return "Pass/Fail System";
10        }
11    }
```

Java

GradeCalculator.java (Uses Strategy Pattern):

```
1     public class GradeCalculator {
2         private GradingStrategy strategy;
3
4         public GradeCalculator(GradingStrategy strategy) {
5             this.strategy = strategy;
6         }
7
8         public void setStrategy(GradingStrategy strategy) {
9             this.strategy = strategy;
10        }
```

Java

```
10     }
11
12     public String calculateGrade(double score) {
13         return strategy.calculateGrade(score);
14     }
15
16     public String getGradingSystemName() {
17         return strategy.getGradingSystemName();
18     }
19 }
```

DesignPatternsDemo.java:

```
1     public class DesignPatternsDemo {
2         public static void main(String[] args) {
3             System.out.println("≡≡≡ Design Patterns Demo ≡≡≡\n");
4
5             // 1. Singleton Pattern
6             System.out.println("--- Singleton Pattern ---");
7             ConfigurationManager config1 = ConfigurationManager.getInstance();
8             ConfigurationManager config2 = ConfigurationManager.getInstance();
9             System.out.println("Same instance? " + (config1 == config2));
10            System.out.println("University: " + config1.getUniversityName());
11            System.out.println("Max Students: " + config1.getMaxStudents());
12
13            // 2. Observer Pattern
14            System.out.println("\n--- Observer Pattern ---");
15            ObservableStudentRegistry registry = new ObservableStudentRegistry();
16            ConsoleLogger logger = new ConsoleLogger();
17            registry.addObserver(logger);
18
19            Student s1 = new Student("Alice", "Smith", 22);
20            registry.addStudent(s1);
21            registry.modifyStudent(s1);
22            registry.removeStudent(s1);
23
24            // 3. Factory Pattern
25            System.out.println("\n--- Factory Pattern ---");
26            Person student = PersonFactory.createPerson("student", "Bob", "Jones",
27                21);
28            Person professor = PersonFactory.createPerson("professor", "Dr. Smith",
29                "Brown", 45);
30            Person staff = PersonFactory.createPerson("staff", "Jane", "Doe", 35);
31
32            student.displayInfo();
33            System.out.println();
34            professor.displayInfo();
35            System.out.println();
```

Java

```
34     staff.displayInfo();
35
36     // 4. Strategy Pattern
37     System.out.println("\n--- Strategy Pattern ---");
38     double score = 85.0;
39
40     GradeCalculator calculator = new GradeCalculator(new
41     GermanGradingStrategy());
42     System.out.println(calculator.getGradingSystemName() + ": " +
43     calculator.calculateGrade(score));
44
45     calculator.setStrategy(new ECTSGradingStrategy());
46     System.out.println(calculator.getGradingSystemName() + ": " +
47     calculator.calculateGrade(score));
48
49     calculator.setStrategy(new PassFailGradingStrategy());
50     System.out.println(calculator.getGradingSystemName() + ": " +
51     calculator.calculateGrade(score));
52 }
```

Task 6: Exception Handling and File I/O

Implement comprehensive error handling and data persistence for your student management system. This task covers exception handling, file operations, and data serialization.

Add robust error handling and file-based data persistence:

- Custom exception classes for domain-specific errors
- File I/O operations for saving and loading student data
- Data validation with appropriate exception handling
- Logging system for tracking system events

Your program should:

1. Create custom exception classes:
 - `StudentNotFoundException` for lookup failures
 - `InvalidMatriculationNumberException` for number conflicts
 - `StudentRegistryFullException` when capacity is exceeded
 - `DataPersistenceException` for file operation errors
2. Implement file-based persistence:
 - Save student data to CSV and JSON formats
 - Load student data from files with error recovery
 - Backup and restore functionality
3. Add comprehensive input validation:
 - Validate name formats, matriculation numbers, dates
 - Handle malformed input gracefully
 - Provide meaningful error messages to users
4. Implement logging system:
 - Log system events, errors, and user actions
 - Different log levels (INFO, WARNING, ERROR)
 - Configurable log output (console, file)

Requirements

- Create custom exception hierarchy with meaningful inheritance
- Use try-with-resources for proper resource management
- Implement both checked and unchecked exceptions appropriately
- Demonstrate exception propagation and handling at different levels
- Use proper file I/O with error recovery mechanisms

Assistance

Custom exception hierarchy:

```
1      public class StudentManagementException extends Exception {
2          public StudentManagementException(String message) {
3              super(message);
4          }
5
6          public StudentManagementException(String message, Throwable cause) {
7              super(message, cause);
8          }
9      }
10
11     public class StudentNotFoundException extends StudentManagementException {
12         public StudentNotFoundException(int matriculationNumber) {
13             super("Student with matriculation number " + matriculationNumber + "
14                 not found");
15         }
16     }
17     public class StudentRegistryFullException extends StudentManagementException {
```

Java

```
18     public StudentRegistryFullException(int maxCapacity) {
19         super("Student registry is full. Maximum capacity: " + maxCapacity);
20     }
21 }
```

File I/O with exception handling:

```
1     public class StudentDataManager { Java
2         public void saveToFile(List<Student> students, String filename)
3             throws DataPersistenceException {
4             try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
5                 writer.println("MatriculationNumber,FirstName,LastName,Age");
6                 for (Student student : students) {
7                     writer.printf("%d,%s,%s,%d%n",
8                         student.getMatriculationNumber(),
9                         student.getFirstName(),
10                        student.getLastName(),
11                        student.getAge());
12                 }
13             } catch (IOException e) {
14                 throw new DataPersistenceException("Failed to save student data",
15                     e);
16             }
17
18             public List<Student> loadFromFile(String filename)
19                 throws DataPersistenceException {
20                 List<Student> students = new ArrayList<>();
21                 try (BufferedReader reader = new BufferedReader(new
22                     FileReader(filename))) {
23                     String line = reader.readLine(); // Skip header
24                     while ((line = reader.readLine()) != null) {
25                         try {
26                             Student student = parseStudentLine(line);
27                             students.add(student);
28                         } catch (IllegalArgumentException e) {
29                             System.err.println("Skipping invalid line: " + line);
30                         }
31                     }
32                 } catch (IOException e) {
33                     throw new DataPersistenceException("Failed to load student data",
34                         e);
35                 }
36                 return students;
37             }
38
39             private Student parseStudentLine(String line) {
40                 String[] parts = line.split(",");
```

```

39         if (parts.length != 4) {
40             throw new IllegalArgumentException("Invalid data format");
41         }
42         // Parse and validate data, throw exceptions for invalid formats
43         return new Student(parts[1], parts[2]);
44     }
45 }

```

Input validation with exceptions:

```

1     public class InputValidator {
2         public static void validateName(String name) throws
3             IllegalArgumentException {
4             if (name == null || name.trim().isEmpty()) {
5                 throw new IllegalArgumentException("Name cannot be empty");
6             }
7             if (!name.matches("[a-zA-ZäöüÄÖÜß\\s-]+")) {
8                 throw new IllegalArgumentException("Name contains invalid
9                 characters");
10            }
11        }
12
13        public static void validateMatriculationNumber(int number)
14            throws InvalidMatriculationNumberException {
15            if (number < 1001 || number > 999999) {
16                throw new InvalidMatriculationNumberException(
17                    "Matriculation number must be between 1001 and 999999");
18            }
19        }
20    }

```

Java

Solution

StudentManagementException.java (Base Exception):

```

1     public class StudentManagementException extends Exception {
2         public StudentManagementException(String message) {
3             super(message);
4         }
5
6         public StudentManagementException(String message, Throwable cause) {
7             super(message, cause);
8         }
9     }

```

Java

StudentNotFoundException.java:

```

1     public class StudentNotFoundException extends
2         StudentManagementException {
3         public StudentNotFoundException(int matriculationNumber) {
4             super("Student with matriculation number " + matriculationNumber + " not
5             found");
6         }
7     }

```

Java

```
4     }  
5     }
```

InvalidMatriculationNumberException.java:

```
1     public class InvalidMatriculationNumberException extends  
    StudentManagementException {  
2         public InvalidMatriculationNumberException(String message) {  
3             super(message);  
4         }  
5     }
```

Java

StudentRegistryFullException.java:

```
1     public class StudentRegistryFullException extends  
    StudentManagementException {  
2         public StudentRegistryFullException(int maxCapacity) {  
3             super("Student registry is full. Maximum capacity: " + maxCapacity);  
4         }  
5     }
```

Java

DataPersistenceException.java:

```
1     public class DataPersistenceException extends  
    StudentManagementException {  
2         public DataPersistenceException(String message) {  
3             super(message);  
4         }  
5  
6         public DataPersistenceException(String message, Throwable cause) {  
7             super(message, cause);  
8         }  
9     }
```

Java

InputValidator.java:

```
1     public class InputValidator {  
2         public static void validateName(String name) throws IllegalArgumentException  
        {  
3             if (name == null || name.trim().isEmpty()) {  
4                 throw new IllegalArgumentException("Name cannot be empty");  
5             }  
6             if (!name.matches("[a-zA-ZäöüÄÖÜß\\s-]+")) {  
7                 throw new IllegalArgumentException("Name contains invalid characters");  
8             }  
9         }  
10  
11        public static void validateMatriculationNumber(int number)  
12            throws InvalidMatriculationNumberException {  
13            if (number < 1001 || number > 999999) {  
14                throw new InvalidMatriculationNumberException(  
15                    "Matriculation number must be between 1001 and 999999");  
16            }  
17        }
```

Java

```
17     }
18
19     public static void validateAge(int age) throws IllegalArgumentException {
20         if (age < 16 || age > 100) {
21             throw new IllegalArgumentException("Age must be between 16 and 100");
22         }
23     }
24 }
```

StudentDataManager.java (File I/O with Exception Handling):

```
1     import java.io.*;
2     import java.util.ArrayList;
3     import java.util.List;
4
5     public class StudentDataManager {
6
7         // Save students to CSV file
8         public void saveToCSV(List<Student> students, String filename)
9             throws DataPersistenceException {
10             try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
11                 // Write header
12                 writer.println("MatriculationNumber,FirstName,LastName,Age");
13
14                 // Write student data
15                 for (Student student : students) {
16                     writer.printf("%d,%s,%s,%d\n",
17                         student.getMatriculationNumber(),
18                         student.getFirstName(),
19                         student.getLastName(),
20                         student.getAge());
21                 }
22
23                 System.out.println("Successfully saved " + students.size() + " students
24                     to " + filename);
25             } catch (IOException e) {
26                 throw new DataPersistenceException("Failed to save student data to " +
27                     filename, e);
28             }
29
30             // Load students from CSV file
31             public List<Student> loadFromCSV(String filename) throws
32                 DataPersistenceException {
33                 List<Student> students = new ArrayList<>();
34                 int lineNumber = 0;
```

Java

```
35     try (BufferedReader reader = new BufferedReader(new
36         FileReader(filename))) {
37         String line = reader.readLine(); // Skip header
38         lineNumber++;
39
40         while ((line = reader.readLine()) != null) {
41             lineNumber++;
42             try {
43                 Student student = parseStudentLine(line);
44                 students.add(student);
45             } catch (IllegalArgumentException e) {
46                 System.err.println("Warning: Skipping invalid line " + lineNumber +
47                     ": " + line);
48                 System.err.println(" Reason: " + e.getMessage());
49             }
50
51             System.out.println("Successfully loaded " + students.size() + "
52                 students from " + filename);
53
54         } catch (FileNotFoundException e) {
55             throw new DataPersistenceException("File not found: " + filename, e);
56         } catch (IOException e) {
57             throw new DataPersistenceException("Failed to load student data from "
58                 + filename, e);
59         }
60
61         return students;
62     }
63
64     // Parse a single CSV line into a Student object
65     private Student parseStudentLine(String line) throws
66         IllegalArgumentException {
67         String[] parts = line.split(",");
68
69         if (parts.length != 4) {
70             throw new IllegalArgumentException("Invalid data format: expected 4
71                 fields, got " + parts.length);
72         }
73
74         try {
75             int matricNo = Integer.parseInt(parts[0].trim());
76             String firstName = parts[1].trim();
77             String lastName = parts[2].trim();
78             int age = Integer.parseInt(parts[3].trim());
79
80             // Validate data
81             InputValidator.validateName(firstName);
```

```

77         InputValidator.validateName(lastName);
78         InputValidator.validateAge(age);
79
80         // Create student (note: this will auto-assign a new matriculation
81         number)
82         return new Student(firstName, lastName, age);
83     } catch (NumberFormatException e) {
84         throw new IllegalArgumentException("Invalid number format in data", e);
85     } catch (Exception e) {
86         throw new IllegalArgumentException("Validation failed: " +
87             e.getMessage(), e);
88     }
89
90     // Backup file
91     public void backup(String sourceFile, String backupFile) throws
92     DataPersistenceException {
93         try (FileInputStream input = new FileInputStream(sourceFile);
94             FileOutputStream output = new FileOutputStream(backupFile)) {
95
96             byte[] buffer = new byte[1024];
97             int bytesRead;
98
99             while ((bytesRead = input.read(buffer)) != -1) {
100                 output.write(buffer, 0, bytesRead);
101             }
102
103             System.out.println("Backup created: " + backupFile);
104
105         } catch (IOException e) {
106             throw new DataPersistenceException("Failed to create backup", e);
107         }
108     }

```

ExceptionHandlingDemo.java:

```

1     import java.util.ArrayList;
2     import java.util.List;
3     import java.util.Scanner;
4
5     public class ExceptionHandlingDemo {
6         public static void main(String[] args) {
7             Scanner scanner = new Scanner(System.in);
8             List<Student> students = new ArrayList<>();
9             StudentDataManager dataManager = new StudentDataManager();
10            String filename = "students.csv";

```

Java

```
11
12     System.out.println("≡≡≡ Student Management with Exception Handling
13     ≡≡\n");
14
15     // Try to load existing data
16     try {
17         students = dataManager.loadFromCSV(filename);
18     } catch (DataPersistenceException e) {
19         System.out.println("Could not load existing data: " + e.getMessage());
20         System.out.println("Starting with empty registry.");
21     }
22
23     boolean running = true;
24
25     while (running) {
26         System.out.println("\n≡≡≡ Menu ≡≡≡");
27         System.out.println("1: Add student");
28         System.out.println("2: Display all students");
29         System.out.println("3: Find student by matriculation number");
30         System.out.println("4: Save to file");
31         System.out.println("5: Load from file");
32         System.out.println("6: Create backup");
33         System.out.println("0: Exit");
34         System.out.print("Your choice: ");
35
36         try {
37             int choice = scanner.nextInt();
38             scanner.nextLine();
39
40             switch (choice) {
41                 case 0:
42                     running = false;
43                     System.out.println("Exiting...");
44                     break;
45
46                 case 1:
47                     // Add student with validation
48                     try {
49                         System.out.print("First name: ");
50                         String firstName = scanner.nextLine();
51                         InputValidator.validateName(firstName);
52
53                         System.out.print("Last name: ");
54                         String lastName = scanner.nextLine();
55                         InputValidator.validateName(lastName);
56
57                         System.out.print("Age: ");
```

```
57         int age = scanner.nextInt();
58         InputValidator.validateAge(age);
59
60         Student student = new Student(firstName, lastName, age);
61         students.add(student);
62         System.out.println("Student added successfully!");
63         System.out.println(student);
64
65     } catch (IllegalArgumentException e) {
66         System.err.println("Validation error: " + e.getMessage());
67     }
68     break;
69
70 case 2:
71     // Display all students
72     if (students.isEmpty()) {
73         System.out.println("No students in registry.");
74     } else {
75         System.out.println("\n≡ All Students ≡");
76         for (Student s : students) {
77             System.out.println(s);
78         }
79     }
80     break;
81
82 case 3:
83     // Find student
84     try {
85         System.out.print("Enter matriculation number: ");
86         int matricNo = scanner.nextInt();
87         InputValidator.validateMatriculationNumber(matricNo);
88
89         Student found = null;
90         for (Student s : students) {
91             if (s.getMatriculationNumber() == matricNo) {
92                 found = s;
93                 break;
94             }
95         }
96
97         if (found != null) {
98             System.out.println("Found: " + found);
99         } else {
100             throw new StudentNotFoundException(matricNo);
101         }
102     }
```

```
103         } catch (InvalidMatriculationNumberException |
104         StudentNotFoundException e) {
105             System.err.println("Error: " + e.getMessage());
106         }
107         break;
108     case 4:
109         // Save to file
110         try {
111             dataManager.saveToCSV(students, filename);
112         } catch (DataPersistenceException e) {
113             System.err.println("Error saving data: " + e.getMessage());
114         }
115         break;
116
117     case 5:
118         // Load from file
119         try {
120             students = dataManager.loadFromCSV(filename);
121         } catch (DataPersistenceException e) {
122             System.err.println("Error loading data: " + e.getMessage());
123         }
124         break;
125
126     case 6:
127         // Create backup
128         try {
129             dataManager.backup(filename, filename + ".backup");
130         } catch (DataPersistenceException e) {
131             System.err.println("Error creating backup: " + e.getMessage());
132         }
133         break;
134
135     default:
136         System.out.println("Invalid choice!");
137     }
138
139 } catch (Exception e) {
140     System.err.println("Unexpected error: " + e.getMessage());
141     scanner.nextLine(); // Clear buffer
142 }
143 }
144
145 // Auto-save on exit
146 try {
147     dataManager.saveToCSV(students, filename);
148     System.out.println("Data saved automatically.");
```

```
149         } catch (DataPersistenceException e) {  
150             System.err.println("Warning: Could not save data on exit: " +  
151                                 e.getMessage());  
152         }  
153         scanner.close();  
154     }  
155 }
```

Lab Execution:

If your program is not yet working without issue, we will try to correct this during the course of the lab. With good preparation, this should not be a problem. Every student is required to be able to explain their thought process at the beginning of the lab. By the end of the lab, the task needs to be completed. Of course, we will support you, but your personal commitment must also be clearly recognizable! Julian Moldenhauer, Furkan Yildirim, and Emily Antosch wish you lots of fun and success!